SOFTWARE REQUIREMENTS SPECIFICATION

DormFix — Hostel Complaint and Maintenance Management System

1. Introduction

1.1 Purpose

DormFix is a web-based hostel complaint and maintenance management system designed to provide a simple and organized way for hostel residents to report maintenance problems and for wardens to monitor and resolve them.

In a hostel environment, issues such as damaged electrical fittings, leaking taps, broken furniture, faulty lights, or other maintenance problems are often reported informally through verbal communication or messages. Such complaints can be difficult to track, and residents may not always know whether their complaint has been received or acted upon.

DormFix provides a centralized system in which a resident can submit a complaint, track its current status, and provide feedback after resolution. The warden can view complaints from residents, prioritize them, update their status, identify complaints that have remained unresolved for more than 48 hours, and monitor overall maintenance activity.

This Software Requirements Specification defines the functional and non-functional requirements of DormFix and serves as the basis for its design, implementation, testing, and evaluation.

1.2 Scope

The scope of DormFix covers the complete lifecycle of a hostel maintenance complaint, beginning with its submission by a resident and ending with its resolution and, where applicable, resident feedback.

The system provides the following major capabilities:

- Resident registration and secure login.
- Warden login and role-based access.
- Submission of hostel maintenance complaints.
- Optional image attachment with a complaint.
- Automatic association of complaints with the resident's registered details.
- Viewing and tracking of complaint status.
- Warden access to complaints submitted by all residents.
- Complaint filtering, sorting, and manual prioritization.
- Updating complaint status.
- Identification of complaints that remain unresolved for more than 48 hours.
- Feedback and rating after a complaint has been resolved.
- Dashboard statistics for monitoring complaint activity.

The current version focuses on complaint reporting and tracking. Technician management, automated technician assignment, online payments, SMS/email notifications, and AI-based complaint prioritization are outside the scope of this version.

1.3 Objectives

The primary objectives of DormFix are to:

1. Provide residents with a convenient method for reporting hostel maintenance problems.
2. Maintain a centralized record of all submitted complaints.
3. Allow residents to track the progress of their complaints.
4. Help wardens organize and prioritize unresolved complaints.
5. Make overdue complaints easily identifiable.
6. Maintain a clear record of complaint resolution and resident feedback.
7. Reduce dependence on informal and untracked complaint-reporting methods.

1.4 Stakeholders

The major stakeholders of DormFix are:

Residents

Residents use the system to report maintenance problems, view the progress of their complaints, and provide feedback after a complaint has been resolved.

Warden

The warden is responsible for monitoring complaints, deciding their priority, updating their status, and ensuring that reported maintenance issues are addressed.

Hostel Administration

Hostel administration can benefit from a structured record of maintenance complaints and their resolution status. The system can provide a clearer overview of recurring and unresolved maintenance issues.

1.5 Definitions and Abbreviations

Term	Definition
Resident	A hostel student who uses DormFix to report and track maintenance complaints.
Warden	The authorized hostel staff member responsible for managing complaints.
Complaint	A maintenance issue reported by a resident through the system.
Status	The current stage of a complaint in its lifecycle.
Overdue Complaint	A complaint that has remained unresolved for more than 48 hours from the time it was submitted.
JWT	JSON Web Token used to authenticate users and maintain secure sessions.

Term	Definition
MERN	MongoDB, Express.js, React, and Node.js technology stack.
Feedback	A rating and optional comment submitted by a resident after resolution of a complaint.

2. Overall Description

2.1 Product Perspective

DormFix is a standalone web application intended to manage hostel maintenance complaints digitally.

The system follows a client-server architecture. The user interacts with a React-based web interface, while the backend provides REST APIs for authentication, complaint management, feedback, and related operations. MongoDB is used to store application data.

The major components of the system are:

- **Resident Interface** — used for registration, login, complaint submission, complaint tracking, and feedback.
- **Warden Interface** — used for viewing, filtering, prioritizing, and resolving complaints.
- **Backend API** — handles authentication, authorization, validation, complaint processing, and business rules.
- **Database** — stores user, complaint, feedback, and related information.

2.2 User Classes

Resident

A resident is an authenticated hostel student who can:

- Create an account.
- Log in to the system.
- Submit maintenance complaints.
- View their submitted complaints.
- Track complaint status.
- View complaint details.
- Submit feedback after resolution.

A resident can access only their own complaints and related information.

Warden

The warden is an authorized hostel administrator who can:

- Log in using a warden account.
- View complaints submitted by residents.
- Filter and sort complaints.

- Manually adjust complaint priority.
- Update complaint status.
- Identify overdue complaints.
- View complaint statistics.

The warden account is provisioned by the system and is not created through normal resident registration.

2.3 System Environment

DormFix is intended to operate in the following environment:

Client side

- Modern web browsers such as Google Chrome, Mozilla Firefox, or Microsoft Edge.
- Desktop and mobile screen sizes.

Server side

- Node.js runtime environment.
- Express.js backend framework.
- MongoDB database.

Development environment

- React with Vite for the frontend.
- Node.js and Express.js for the backend.
- MongoDB for persistent data storage.

2.4 System Constraints

The following constraints apply to the current version of DormFix:

1. The system shall be developed using the MERN stack.
2. User passwords shall not be stored in plaintext.
3. Access to protected resources shall require authentication.
4. Access to system functionality shall depend on the user's role.
5. A resident shall not be able to access another resident's complaints.
6. Uploaded complaint images shall be limited to supported image formats and file sizes.
7. Complaint priority shall be determined manually by the warden.
8. The current version shall not depend on paid third-party services for its core functionality.

2.5 Assumptions and Dependencies

The system is developed under the following assumptions:

- Each resident has one active DormFix account.
- Each resident is associated with one hostel room.
- A resident's registration number is unique.
- A resident's email address is unique.
- An authorized warden account is available for managing complaints.
- The system has access to a functioning MongoDB database.

- Users have access to a modern web browser and a network connection.
- The hostel staff using the system are responsible for acting on complaints after they are reported.

3. System Features and Functional Requirements

This section describes the functional requirements of DormFix. Each requirement has a unique identifier so that it can be traced during implementation and testing.

3.1 User Registration and Authentication

ID	Functional Requirement	Priority
FR-01	The system shall allow a resident to create an account by providing: - Full name - Registration number - Room number - Email address - Password	High
FR-02	The system shall prevent registration when the provided email address or registration number is already associated with an existing account.	High
FR-03	The system shall allow registered residents and authorized wardens to log in using their email address and password.	High
FR-04	After successful authentication, the system shall issue a JWT that is used to authenticate subsequent requests to protected resources.	High
FR-05	The system shall identify whether an authenticated user is a resident or warden and shall provide access only to functionality permitted for that role.	High
FR-06	The system shall allow a logged-in user to log out and terminate the active client-side session.	Medium

3.2 Complaint Management

ID	Functional Requirement	Priority
FR-07	The system shall allow a resident to submit a maintenance complaint containing a description of the problem and an optional image.	High
FR-08	The system shall associate the complaint with the authenticated resident's name, registration number, and room number obtained from the user's account rather than relying on information manually entered in the complaint form.	High
FR-09	The system shall assign each complaint a unique identifier for future reference. The identifier may follow a format such as:	Medium

ID	Functional Requirement	Priority
	HF-0001, HF-0002, HF-0003, ...	
FR-10	The system shall assign the status **SUBMITTED** to every newly created complaint.	High
FR-11	The system shall allow a resident to view a list of complaints submitted through their account.	High
FR-12	The system shall allow a resident to view the complete details of an individual complaint submitted by them.	High
FR-13	The system shall restrict residents to viewing their own complaints and shall prevent access to complaints belonging to other residents.	High

3.3 Complaint Status Management

A complaint shall follow the defined lifecycle:

SUBMITTED → UNDER_PROGRESS → RESOLVED

The system may also allow a direct transition:

SUBMITTED → RESOLVED

ID	Functional Requirement	Priority
FR-14	The system shall allow the warden to update a complaint's status according to the defined complaint lifecycle.	High
FR-15	The system shall reject status changes that do not comply with the defined status transition rules.	High
FR-16	When a complaint is marked as resolved, the system shall record the relevant resolution time required for complaint tracking and statistics.	High

3.4 Warden Complaint Management

ID	Functional Requirement	Priority
FR-17	The system shall allow the warden to view complaints submitted by all residents.	High
FR-18	The system shall allow the warden to filter complaints based on relevant attributes such as: - Complaint status - Overdue status - Room number	Medium

ID	Functional Requirement	Priority
FR-19	The system shall allow the warden to sort complaints using criteria such as: - Newest first - Oldest first - Status - Resolution date - Overdue complaints first	Medium
FR-20	The system shall allow the warden to manually move complaints up or down in the priority order. The system shall not automatically assign complaint priority using AI or an automated urgency-scoring mechanism.	Medium
FR-21	The system shall identify a complaint as overdue when it has remained unresolved for more than 48 hours from its creation time. The overdue status shall be determined by the server using the complaint's recorded creation time.	High

3.5 Feedback Management

ID	Functional Requirement	Priority
FR-22	The system shall allow a resident to provide feedback for a complaint after the complaint has been marked as resolved. Feedback shall contain: - A rating from 1 to 5. - An optional comment.	High
FR-23	The system shall prevent residents from submitting feedback for complaints that have not yet been resolved.	High
FR-24	The system shall allow a resident to submit feedback only once for a particular complaint.	Medium

3.6 Dashboard and Statistics

ID	Functional Requirement	Priority
FR-25	The system shall provide the warden with summarized complaint information, including: - Total complaints - Submitted complaints	Medium

ID	Functional Requirement	Priority
	- Complaints under progress - Resolved complaints - Overdue complaints - Average resolution time The statistics shall be generated from the complaint records maintained by the system.	

4. Non-Functional Requirements

4.1 Security Requirements

ID	Non-Functional Requirement	Category
NFR-01	User passwords shall be hashed using a secure password-hashing mechanism such as bcrypt before being stored in the database.	Security
NFR-02	Protected API endpoints shall verify the identity and role of the requesting user before providing access to protected data or operations.	Security
NFR-03	A resident shall not be able to view, modify, or access another resident's complaints, even if the identifier of another complaint is known.	Security
NFR-04	API responses shall not expose stored passwords or password-related fields.	Security
NFR-05	The system shall validate user input on the server side before storing or processing it.	Security

4.2 File Upload Requirements

ID	Non-Functional Requirement	Category
NFR-06	Complaint images shall be restricted to supported formats such as JPG, PNG, and WEBP.	Security / Usability
NFR-07	The maximum size of an uploaded complaint image shall be 5 MB.	Security / Usability

4.3 Performance Requirements

ID	Non-Functional Requirement	Category
NFR-08	Under normal development/demo load, typical API requests should be processed and returned within approximately 500 milliseconds, excluding delays caused by network conditions or external services.	Performance

4.4 Usability Requirements

ID	Non-Functional Requirement	Category
NFR-09	The user interface shall remain usable across common desktop and mobile screen sizes.	Usability
NFR-10	The system shall provide clear loading, success, empty, and error states so that users can understand the current state of an operation.	Usability
NFR-11	The system shall provide separate and understandable interfaces for residents and wardens, with functionality relevant to each user's role.	Usability

4.5 Maintainability Requirements

ID	Non-Functional Requirement	Category
NFR-12	The application shall follow a modular structure separating major responsibilities such as routes, controllers, services, models, and frontend components.	Maintainability
NFR-13	The codebase shall follow consistent naming, organization, and error-handling practices to facilitate future maintenance.	Maintainability
NFR-14	The system shall provide a seed mechanism through which a known demonstration dataset and required initial accounts can be recreated when necessary.	Maintainability

5. External Interface Requirements

5.1 User Interface

The system shall provide web interfaces appropriate to each user role.

Resident Interface

The resident interface shall provide access to:

- Registration
- Login
- Complaint submission
- Complaint list
- Complaint details
- Complaint status
- Feedback submission
- Logout

Warden Interface

The warden interface shall provide access to:

- Login
- Complaint dashboard
- Complaint filtering
- Complaint sorting
- Complaint priority management
- Status updates
- Overdue complaint identification
- Complaint statistics
- Logout

5.2 Software Interfaces

The system shall interact with:

- MongoDB for persistent data storage.
- Node.js and Express.js for backend services.
- React for the web-based frontend.

5.3 Communication Interface

The frontend shall communicate with the backend through HTTP/HTTPS-based REST API requests.

Protected requests shall include the required authentication information.

---

6. Data Requirements

DormFix shall maintain information required for user authentication, complaint management, and feedback.

6.1 Resident Data

The system shall maintain information such as:

- Name
- Registration number
- Room number
- Email address
- Hashed password
- User role

6.2 Complaint Data

The system shall maintain information such as:

- Complaint ID
- Resident reference
- Room number

- Complaint description
- Optional image
- Creation time
- Current status
- Priority information
- Resolution time

6.3 Feedback Data

The system shall maintain:

- Complaint reference
- Resident reference
- Rating
- Optional comment
- Feedback submission time

The system shall ensure that each complaint can have at most one feedback record from its resident.

---

7. Business Rules

The following rules define important operational behaviour of DormFix:

BR-01: Only authenticated residents can submit complaints.

BR-02: A resident can view only complaints associated with their own account.

BR-03: Only an authenticated warden can view and manage complaints across residents.

BR-04: A newly submitted complaint starts with the status **SUBMITTED**.

BR-05: A complaint can move from **SUBMITTED** to **UNDER_PROGRESS** or directly to **RESOLVED**.

BR-06: A complaint marked **UNDER_PROGRESS** can be moved to **RESOLVED**.

BR-07: Invalid status transitions shall be rejected by the system.

BR-08: A complaint becomes overdue when it remains unresolved for more than 48 hours from its creation time.

BR-09: Feedback can be submitted only after a complaint has been resolved.

BR-10: A resident can submit feedback only once for a particular complaint.

BR-11: Complaint priority is determined manually by the warden.

---

8. Requirement Prioritization

The requirements are prioritized according to their importance to the core operation of DormFix.

Must Have

The following functionality is essential to the operation of the system:

- Resident registration and authentication
- Warden authentication
- Complaint submission
- Complaint tracking
- Complaint status management
- Role-based access control
- Complaint ownership and data isolation
- Overdue complaint identification
- Feedback after resolution
- Secure password storage

Should Have

The following functionality significantly improves the usability and management of the system:

- Complaint filtering
- Complaint sorting
- Manual priority management
- Dashboard statistics
- Responsive interface
- Image upload restrictions
- Clear loading and error states

Could Have

Possible enhancements for future versions include:

- Drag-and-drop complaint prioritization.
- More detailed analytics and reporting.
- Complaint categories and recurring-issue analysis.
- Additional administrative roles.

Will Not Be Included in the Current Version

The following features are outside the scope of the current system:

- Technician accounts and assignment.
- Online payment processing.
- Automated SMS/email notifications.
- AI-based complaint prioritization.
- Integration with existing college/hostel management systems.

---

9. Acceptance Criteria

DormFix shall be considered to satisfy a requirement when the corresponding functionality:

1. Is implemented according to the specified requirement.
2. Produces the expected result under normal operating conditions.
3. Handles relevant invalid input or unauthorized access appropriately.
4. Can be verified through a suitable manual or automated test case.

Examples of important acceptance scenarios include:

- A new resident can successfully create an account using valid information.
- Duplicate email addresses or registration numbers are rejected.
- An authenticated resident can submit a complaint.
- A resident cannot access another resident's complaint.
- A warden can view complaints from multiple residents.
- A complaint follows only the permitted status transitions.
- A complaint that remains unresolved for more than 48 hours is identified as overdue.
- Feedback cannot be submitted before a complaint is resolved.
- A resident cannot submit feedback more than once for the same complaint.
- Passwords are not returned through API responses.

---

10. Future Enhancements

The current version of DormFix establishes the basic complaint management workflow. Future versions may extend the system with:

- Technician or maintenance staff accounts.
- Automatic assignment of complaints to maintenance personnel.
- Email or SMS notifications.
- Complaint categories and severity levels.
- Detailed maintenance analytics.
- Recurring complaint detection.
- Integration with hostel or college administration systems.
- Mobile application support.

These enhancements are not required for the current version but may improve the usefulness of the system when deployed on a larger scale.

---

11. Conclusion

DormFix provides a structured approach to managing hostel maintenance complaints by replacing informal complaint reporting with a centralized digital system. Residents can report and track maintenance issues, while wardens can monitor complaints, manage their priority, update their status, and identify unresolved issues.

The requirements specified in this document define the expected behaviour, constraints, security considerations, and quality requirements of the current version of the system. They provide a common reference for system design, development, testing, and future enhancement.